



SoundForge Challenge — Helpful Resources & Cheat Sheet

Quick references and code patterns to unblock you

Contents

How to use this sheet	1
Where to learn (free, reliable)	2
HTML — a semantic skeleton	2
CSS layout patterns	2
Responsive design	3
Using the design tokens (CSS variables)	4
Placeholder images	4
Git cheat sheet	4
If you get stuck	5

How to use this sheet

This is here to **unblock you, not to do the task for you**. Every code snippet below is a *generic* example (a profile card, a demo nav) — copy the **technique**, then build the actual SoundForge sections yourself from the design spec. Using these patterns is expected and fine; copying a finished landing page is not.

Where to learn (free, reliable)

- **MDN — HTML elements:** <https://developer.mozilla.org/en-US/docs/Web/HTML/Element>
- **MDN — CSS reference:** <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference>
- **CSS-Tricks — A Complete Guide to Flexbox:** <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>
- **CSS-Tricks — A Complete Guide to Grid:** <https://css-tricks.com/snippets/css/complete-guide-grid/>
- **web.dev — Learn Responsive Design:** <https://web.dev/learn/design/>
- **MDN — Using media queries:** https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_media_queries/Using_media_queries
- **GitHub Docs — Get started:** <https://docs.github.com/en/get-started>
- **freeCodeCamp — Responsive Web Design (interactive):** <https://www.freecodecamp.org/learn/2022/responsive-web-design/>

Tip: when you're stuck on a property, search "MDN " — the MDN page almost always has a small live example.

HTML — a semantic skeleton

Use meaningful tags rather than div for everything. A typical page outline:

```
<header>                <!-- logo + nav -->
  <nav> ... </nav>
</header>

<main>
  <section id="about"> ... </section>
  <section id="schedule"> ... </section>
</main>

<footer> ... </footer>
```

Handy elements: <h1>-<h3> for headings (one <h1> per page), for in-page links, / for lists, <table> for tabular data, <button> or for calls to action.

CSS layout patterns

Flexbox — a row with items pushed apart (great for a nav bar)

```
.nav {
  display: flex;
  align-items: center;           /* vertical centring */
  justify-content: space-between; /* logo left, links right */
  gap: 16px;
}
.nav ul { display: flex; gap: 16px; }
```

Flexbox — centre something

```
.hero {
  display: flex;
  flex-direction: column;
  align-items: center;    /* horizontal centre */
  text-align: center;
}
```

Grid — a responsive card row (wraps automatically)

```
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 24px;
}
```

auto-fit + minmax means: fit as many columns as will fit at 220px+, and stack to one column on narrow screens — without writing a media query. You can also set the column count explicitly per breakpoint (see below).

Responsive design

Mobile-first media queries

Write your base styles for small screens, then add overrides as the screen grows:

```
/* base = mobile: single column */
.cards { grid-template-columns: 1fr; }

/* tablet and up */
@media (min-width: 768px) {
  .cards { grid-template-columns: 1fr 1fr; }
}

/* desktop and up */
@media (min-width: 1120px) {
  .cards { grid-template-columns: repeat(3, 1fr); }
}
```

Stop horizontal scrolling on mobile

The usual culprits and fixes:

- Use box-sizing: border-box; so padding doesn't add to width (already set for you in the starter reset).
- Make media flexible: img { max-width: 100%; } (also in the starter).
- Cap content width and centre it: max-width: 1120px; margin-inline: auto;.
- Avoid fixed pixel widths wider than the screen; prefer %, max-width, or grid/flex.

Test it

Open your browser DevTools (**F12**), click the **device toolbar** (phone/tablet icon), and check **375 px**, **768 px**, and **1280 px**. Resizing the window slowly also reveals where the layout breaks.

Using the design tokens (CSS variables)

The starter already defines variables like `--color-primary` and `--space-4`. Use them with `var(...)` instead of hard-coding values:

```
.button {  
  background: var(--color-primary);  
  color: #fff;  
  padding: var(--space-3) var(--space-4);  
  border-radius: 999px;  
}  
.button:hover { background: var(--color-primary-hover); }
```

This keeps everything consistent and makes the page match the spec.

Placeholder images

You don't need real photos. Use a placeholder service or a coloured box:

``

Format: `https://placeholder.co/<width>x<height>/<bg>/<text-colour>?text=<label>`.

Git cheat sheet

```
git status          # what's changed / staged  
git add .           # stage all changes  
git commit -m "message" # save a snapshot with a message  
git log --oneline    # see your commit history
```

connect to your new (empty) GitHub repo and push:

```
git remote add origin https://github.com/<you>/soundforge-challenge.git  
git push -u origin main
```

Common fixes:

- **Wrong last commit message?** `git commit --amend -m "better message"` (before pushing).
- **Forgot to stage a file?** `git add <file>` then commit again.
- **Default branch is master, not main?** Rename it: `git branch -M main`.
- **"remote origin already exists"? Update it:** `git remote set-url origin <url>`.

Aim for several small, clearly-named commits (e.g. "Add nav", "Style hero", "Make cards responsive") rather than one big commit.

If you get stuck

Spend a few minutes searching MDN or the guides above. If you're still blocked, **commit what you have**, write a note in `NOTES.md` about where you got stuck, and submit that — an honest partial attempt is genuinely useful to us and far better than nothing.